

Lattice-Embedding + Tiling Discussion

Algorithms and Implementation

ABHIRUP MUKHERJEE

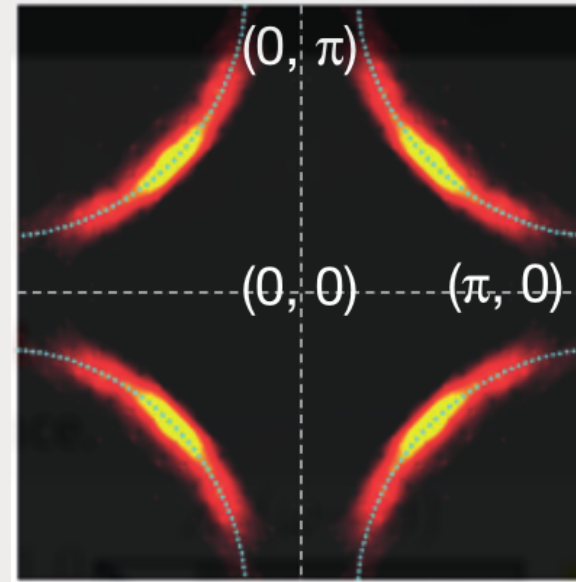
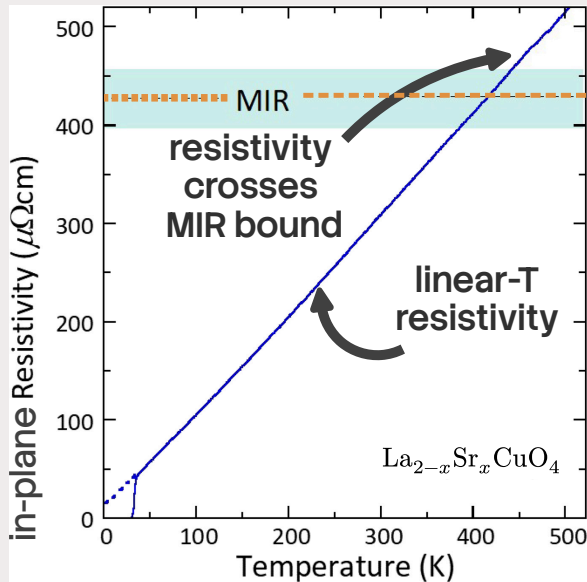


EMERGENT PHENOMENA IN QUANTUM MATTER GROUP
IISER KOLKATA



What's This About?

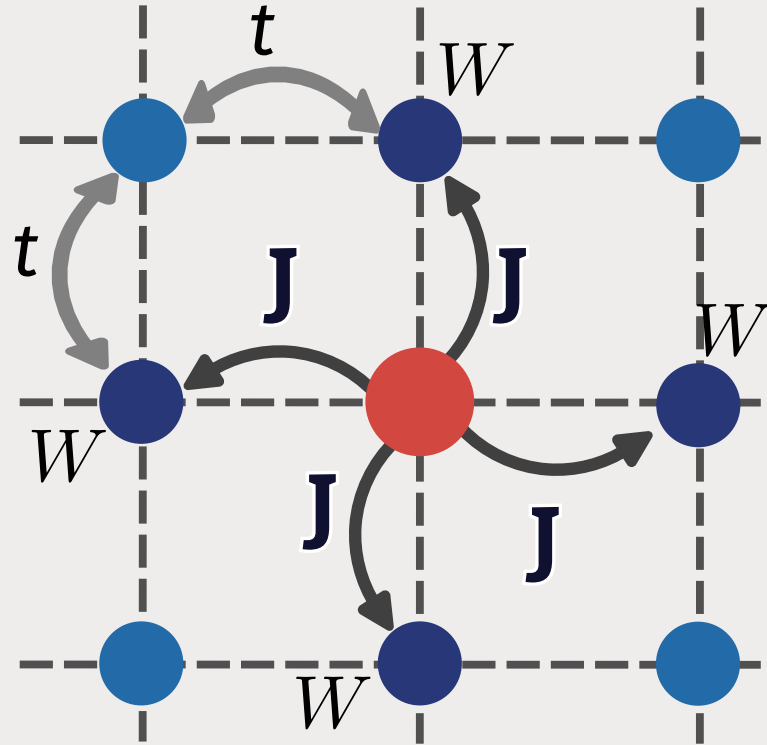
The Problem: 2D Mott MIT At 1/2-Filling



Required: Development of a new auxiliary model method that incorporates

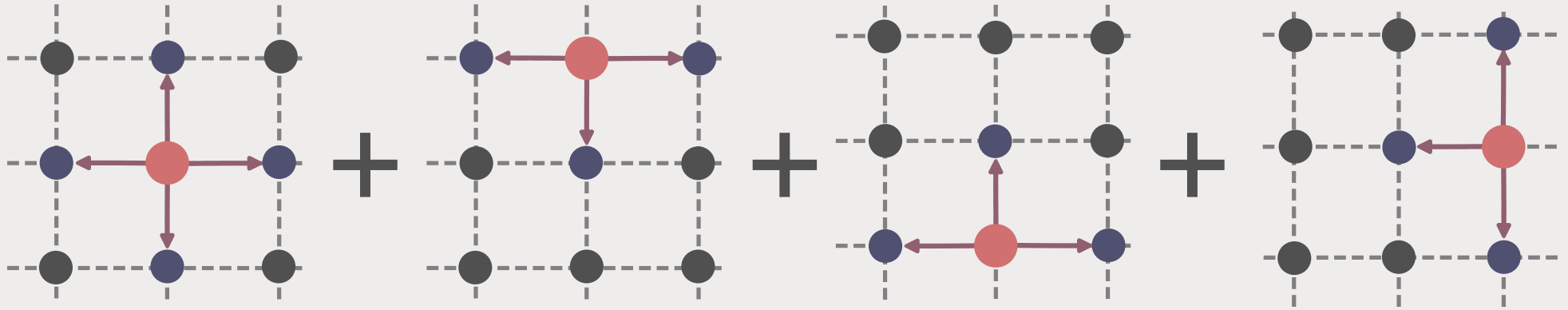
- non-local effects of correlation
- lattice effects

Component 1: Lattice-Embedding



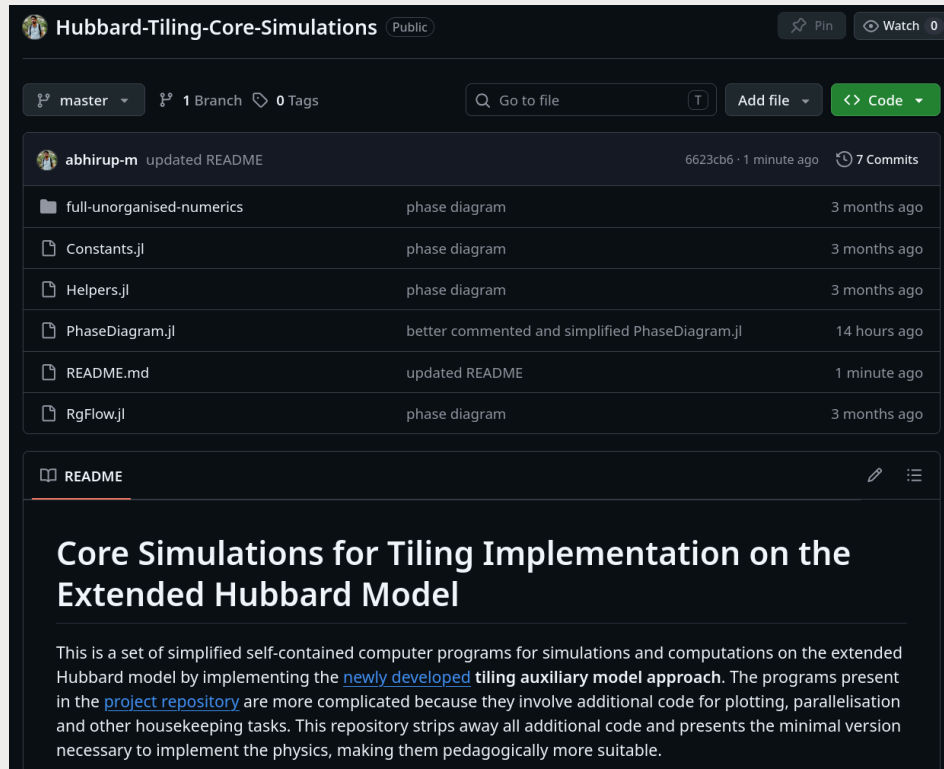
- Impurity dynamics must be solved
- Momentum-dependence of conduction bath quantities must be taken into account

Component 2: Tiling Of Impurity Quantities



Translate impurity model quantities to obtain lattice quantities

Pedagogical Repository



Hubbard-Tiling-Core-Simulations Public

master 1 Branch 0 Tags

Go to file Add file Code

abhirup-m updated README 6623cb6 · 1 minute ago 7 Commits

full-unorganised-numerics	phase diagram	3 months ago
Constants.jl	phase diagram	3 months ago
Helpers.jl	phase diagram	3 months ago
PhaseDiagram.jl	better commented and simplified PhaseDiagram.jl	14 hours ago
README.md	updated README	1 minute ago
RgFlow.jl	phase diagram	3 months ago

README

Core Simulations for Tiling Implementation on the Extended Hubbard Model

This is a set of simplified self-contained computer programs for simulations and computations on the extended Hubbard model by implementing the [newly developed tiling auxiliary model approach](#). The programs present in the [project repository](#) are more complicated because they involve additional code for plotting, parallelisation and other housekeeping tasks. This repository strips away all additional code and presents the minimal version necessary to implement the physics, making them pedagogically more suitable.

Simplified Codes

<https://github.com/abhirup-m/Hubbard-Tiling-Core-Simulations>

- “Extra” stuff removed
- Better commented

Setting Up The Project

Define Some Constants

```
const K_MIN = -pi
const K_MAX = pi
const TOLERANCE = 1e-8
const HOP_T = 1.0
const OMEGA_BY_t = -2
const NODAL_POINTS = [(-pi/2, -pi/2), (-pi/2, pi/2), (pi/2, pi/2), (pi/2, -pi/2)]
const ANTINODAL_POINTS = [(-pi, 0.), (0., pi), (pi, 0.), (0., -pi)]
const RG_RELEVANCE_TOL = 1e-3
```

julia

Define Some Constants

```
const K_MIN = -pi
const K_MAX = pi
const TOLERANCE = 1e-8
const HOP_T = 1.0
const OMEGA_BY_t = -2
const NODAL_POINTS = [(-pi/2, -pi/2), (-pi/2, pi/2), (pi/2, pi/2), (pi/2, -pi/2)]
const ANTINODAL_POINTS = [(-pi, 0.), (0., pi), (pi, 0.), (0., -pi)]
const RG_RELEVANCE_TOL = 1e-3
```

julia

- Keep unchanged throughout all calculations
- Try to keep them dimensionless

Working In Momentum Space: Variables

- Each $(k_x, k_y) \rightarrow m$
- Store single momentum-dependent variable as vector. Dispersion, density of states:

$$E_{k_x, k_y} \longrightarrow E[m]$$

Working In Momentum Space: Variables

- Each $(k_x, k_y) \rightarrow m$
- Store single momentum-dependent variable as vector. Dispersion, density of states:

$$E_{k_x, k_y} \longrightarrow E[m]$$

- Store two momentum-dependent variable as matrix. Kondo coupling:

$$J_{k_1, k_2} \longrightarrow J[m, n]$$

Working In Momentum Space: Variables

- Each $(k_x, k_y) \rightarrow m$
- Store single momentum-dependent variable as vector. Dispersion, density of states:

$$E_{k_x, k_y} \longrightarrow E[m]$$

- Store two momentum-dependent variable as matrix. Kondo coupling:

$$J_{k_1, k_2} \longrightarrow J[m, n]$$

- Store four momentum-dependent variable as function. Call when needed. Bath interaction:

$$W_{k_1, k_2, k_3, k_4} \longrightarrow W(\dots k)$$

Working In Momentum Space: Helpers

- 1D representation to 2D representation:

```
map1DTo2D(m) --> (kx, ky)
```

julia

- Inverse transformation:

```
map2DTo1D(kx, ky) --> m
```

julia

Working In Momentum Space: Helpers

- 1D representation to 2D representation:

```
map1DTo2D(m) --> (kx, ky)
```

julia

- Inverse transformation:

```
map2DTo1D(kx, ky) --> m
```

julia

- Extract points that lie on a certain energy:

```
getIsoEngCont(dispersion, E) --> [m1, m2, ..., mN]
```

julia

(loop through points, check which energies are equal to probe energy upto some tolerance)

Density Of States

- Dispersion is easy

```
E[m] = Dispersion(kx, ky)
```

```
julia
```

- DOS:

```
rho[m] ~ 1/Delta(E[m]) ~ 1/(E[m+1] - E[m])
```

```
julia
```

Density Of States

- Dispersion is easy

```
E[m] = Dispersion(kx, ky)
```

julia

- DOS:

```
rho[m] ~ 1/Delta(E[m]) ~ 1/(E[m+1] - E[m])
```

julia

- Take care of infinities (replace with largest finite value?!)
- Normalise per some convention to keep things bounded

$$\int dE \rho(E) \sim N$$

Renormalisation Group Equation Simulation

RG Equation

$$\Delta J[k_1, k_2] \sim \sum_q (J[k_1, q]J[q, k_2] - 4J[q, \bar{q}]W[k_1, k_2])G[q]$$

RG Equation

$$\Delta J[k_1, k_2] \sim \sum_q (J[k_1, q]J[q, k_2] - 4J[q, \bar{q}]W[k_1, k_2])G[q]$$

Write as matrix equation for vectorisation:

$$\Delta J \sim JG_{UV}J^\dagger - 4W \text{Tr}(\bar{J}G_{UV})$$

where

$$J \longrightarrow J[k_1, k_2], \quad G_{UV} \longrightarrow G[q], \quad W \longrightarrow W[k_1, k_2], \quad \bar{J} = J[q, \bar{q}]$$

RG Flow Simulation: Initialisation

$$\Delta J \sim JG_{UV}J^\dagger - 4W \text{Tr}(\bar{J}G_{UV})$$

- Define dispersion and density of states
- Define bath interaction matrix and initial Kondo matrix
- Extract cutoff energies from dispersion (by varying k_x from $0 \rightarrow \pi$ keeping $k_y = 0$)

RG Flow Simulation: Iteration

$$\Delta J \sim J G_{UV} J^\dagger - 4W \text{Tr}(\bar{J} G_{UV})$$

At each step,

- extract UV and IR momentum points
-

RG Flow Simulation: Iteration

$$\Delta J \sim J G_{UV} J^\dagger - 4W \text{Tr}(\bar{J} G_{UV})$$

At each step,

- extract UV and IR momentum points
- define propagator diagonal matrix G by setting only UV points

```
GMatrix[q, q] = densityOfStates[q] ./ (OMEGA_BY_t * HOP_T -  
energyCutoff / 2 + kondoJArray[q, q] / 4 + bathW / 2)
```

julia

RG Flow Simulation: Iteration

$$\Delta J \sim J G_{UV} J^\dagger - 4W \text{Tr}(\bar{J} G_{UV})$$

At each step,

- extract UV and IR momentum points
- define propagator diagonal matrix G by setting only UV points

```
GMatrix[q, q] = densityOfStates[q] ./ (OMEGA_BY_t * HOP_T -  
energyCutoff / 2 + kondoJArray[q, q] / 4 + bathW / 2)
```

julia

- use matrix multiplication to calculate ΔJ

```
delta = -abs(deltaEnergy) * (J[IR, UV] * G[UV, UV] * J[UV, IR] .- 4 *  
bathW * traceGprime .* WMatrix[IR, IR])
```

julia

RG Flow Simulation: Iteration

$$\Delta J \sim J G_{UV} J^\dagger - 4W \text{Tr}(\bar{J} G_{UV})$$

At each step,

- extract UV and IR momentum points
- define propagator diagonal matrix G by setting only UV points

```
GMatrix[q, q] = densityOfStates[q] ./ (OMEGA_BY_t * HOP_T -  
energyCutoff / 2 + kondoJArray[q, q] / 4 + bathW / 2)
```

julia

- use matrix multiplication to calculate ΔJ

```
delta = -abs(deltaEnergy) * (J[IR, UV] * G[UV, UV] * J[UV, IR] .- 4 *  
bathW * traceGprime .* WMatrix[IR, IR])
```

julia

(Use broadcasting to make the syntax readable)

RG Flow Simulation: Completion

Stop when

- all matrix elements have vanished,
- denominator has changed sign,
- or arrived at Fermi surface.

RG Flow Simulation: Phase Diagram

Multiple kinds of **phase diagram** possible: couplings, correlations, exponents, etc

RG Flow Simulation: Phase Diagram

Multiple kinds of **phase diagram** possible: couplings, correlations, exponents, etc

Fixed point coupling phase diagram

- Simulate RG flow at multiple parameter points (W , J)

```
J = runRGFlow(parameters...)
```

julia

RG Flow Simulation: Phase Diagram

Multiple kinds of **phase diagram** possible: couplings, correlations, exponents, etc

Fixed point coupling phase diagram

- Simulate RG flow at multiple parameter points (W , J)

```
J = runRGFlow(parameters...)
```

julia

- At each fixed point, **count** number of Fermi surface points that are “gapless”

```
poles = 0
for point in fermiSurface
    if abs(J[point, point]) > 0
        poles += 1
    end
end
```

julia

RG Flow Simulation: Phase Diagram

Multiple kinds of **phase diagram** possible: couplings, correlations, exponents, etc

Fixed point coupling phase diagram

- Simulate RG flow at multiple parameter points (W , J)

```
J = runRGFlow(parameters...)
```

julia

- At each fixed point, **count** number of Fermi surface points that are “gapless”

```
poles = 0
for point in fermiSurface
    if abs(J[point, point]) > 0
        poles += 1
    end
end
```

julia

- Plot this count as a function of J and W

Notes

- Parallelisation not helpful within a single RG flow simulation but can speedup multiple ones
- Using reflection and particle-hole symmetries might help but makes code complicated (measure tradeoffs before finalising)
- Choosing system size as $N = 4m + 5$ helps (makes sure nodes, antinodes and midway point is available)

THANK YOU